



*Hardware, registers, memory, and programming
the 65816 accelerator for the Commodore 64 and 128*

A consolidated reference covering the SuperCPU 64 and 128 accelerator cartridges, the SuperRAM expansion card, the WDC 65C816 processor as CMD deployed it, and the behaviours that determine whether C64 software runs on an accelerated machine. Compiled from CMD documentation, the Go64! tutorial series, VICE's SCPU64 implementation, contemporary *Commodore World* articles, and original measurements taken on physical hardware.

Table of Contents

Part I • The Device

- 1 What the SuperCPU is
- 2 How it takes over the machine
- 3 Speed, and the three requests for 1 MHz

Part II • Memory

- 4 The 24-bit address space
- 5 Bank 0 and the C64 PLA
- 6 The bank-1 ROM shadow
- 7 The private RAM windows

Part III • Registers

- 8 Write-sensitive switches, `$D07x`
- 9 The status block, `$D0B0-$D0BF`
- 10 Detecting the accelerator

Part IV • Programming the 65816

- 11 Processor modes
- 12 Registers the 6502 did not have
- 13 Addressing modes and instructions
- 14 Interrupts
- 15 Block moves
- 16 Running code outside bank 0
- 17 Address wrapping — the complete rules

Part V • Writing It in Assembly

- 18 Addressing-mode notation
- 19 The width problem
- 20 Forcing the addressing mode
- 21 Directives and program structure
- 22 Idioms and traps
- 23 Worked examples

Part VI • Driving It from BASIC

- 24 Detection and status
- 25 Speed control, and measuring it
- 26 What BASIC cannot do

Part VII • Compatibility

- 27 The nine rules

- 28 Why acceleration sometimes buys nothing
- 29 Repairing existing software
- 30 The host machine: power, cartridges and companions

Part VIII • The SuperRAM Card

- 31 The card and its SIMM
- 32 Detecting and allocating expansion RAM

Part IX • The SuperCPU 128

- 33 Version 2 architecture
- 34 The enhanced optimization register
- 35 C128 access-class timing rules
- 36 Startup and MMU constraints
- 37 The MMU adapter

Part X • Measured Hardware Behaviour

- 38 The C128 under sustained DMA
- 39 Retention and decay
- 40 Methodology that proved necessary

Part XI • Toward an FPGA Reimplementation

- 41 Why it has not been done
- 42 The specification, as it now stands
- 43 Hardware envelope
- 44 The hard parts
- 45 A staged plan

Appendices

- A Timing and raster arithmetic
- B SuperCPU DOS and the startup animation
- C Sources, and what this document omits

The Device

A SuperCPU does not help the 6510 go faster. It replaces it, and then spends most of its engineering effort pretending it did not.

1. What the SuperCPU is

The CMD SuperCPU is an accelerator cartridge for the Commodore 64 (SuperCPU 64) and Commodore 128 (SuperCPU 128). It plugs into the expansion port and substitutes a Western Design Center 65C816 running at 20 MHz for the machine's own 1 MHz 6510 or 8502.

Component	Specification
Processor	WDC 65C816S at 20 MHz, 8/16-bit, 24-bit address bus
Opcodes	Documented 6510 opcodes only — the undocumented encodings are reused for real 65816 instructions
On-board SRAM	128 KB (SuperCPU 64) · 256 KB (SuperCPU 128 v2)
ROM	SuperCPU DOS — a modified KERNAL and BASIC with JiffyDOS. Circulating dumps are 128 KB.
Expansion	SuperRAM card: 1–16 MB via one 72-pin fast-page-mode SIMM
Glue logic	Altera CPLD; the larger V2 part fixed most of V1's bugs

2. How it takes over the machine

The SuperCPU asserts `/DMA` on the expansion port. The 6510 tri-states its address, data and R/W pins and stops driving the bus; the SuperCPU drives it instead. The VIC-II is untouched throughout — it keeps generating the display, refreshing DRAM and raising raster interrupts.

Three consequences follow, and between them they explain almost every design decision in the product.

CONSEQUENCE 1 — THE I/O PORT IS NO LONGER THE AUTHORITY

The 6510's on-chip port at `$00 / $01` decides C64 banking, and it is inside a chip that is now inert. The accelerator must emulate the port and resolve the memory map itself.

CONSEQUENCE 2 — C64 DRAM GOES STALE

The program reads the accelerator's own fast copy, but the VIC-II reads DRAM. Any write the VIC could observe has to be mirrored back at a cost of one C64 cycle each. This is what the *optimization modes* exist to reduce.

CONSEQUENCE 3 — ULTIMAX CARTRIDGES ARE INCOMPATIBLE

A freezer that pulls `/GAME` low with `/EXROM` high replaces the KERNAL and leaves the accelerator inoperable. CMD's documentation names Action Replay and Super Snapshot specifically.

3. Speed, and the three requests for 1 MHz

20 MHz in turbo, dropping to 1 MHz — or 2 MHz on a C128 in fast mode — when software asks or when the access requires the C64 itself.

CMD's own headline was that a SuperCPU could be **up to forty times** faster than a stock machine when programmed for it — not twenty. The extra factor is not clock speed but the 65816 itself: 16-bit registers roughly halve the instruction count for work that was being done a byte at a time, so well-written native code beats the raw clock ratio. Unmodified C64 software gets the twenty, minus whatever it spends on the real bus.

READS AND WRITES ARE NOT SYMMETRIC

The division of labour is worth stating in one line, because every performance characteristic follows from it: **reads come from the accelerator's own RAM at full speed, while writes go to both the accelerator's RAM and the C64's RAM.**

So reading is free and writing is not. A write costs a real 1 MHz bus cycle because the VIC-II has to be able to see the result, which is why heavy write traffic slows a program down and why the optimization modes of §10 exist at all.

What no manual states, and what only emerges from reading a faithful implementation, is that there is no single turbo flag. There are **three independent requests for 1 MHz**: a system request, a software request, and the physical switch on the cartridge. The machine runs fast only when none of them is asserting.

```
fast = !( sys1MHz || soft1MHz || ( switchSlow && !hwRegsEnabled ) )
```

THE SWITCH IS A PERMISSION, NOT A COMMAND

Note the last term. **The physical speed switch is honoured only while the hardware registers are disabled.** Software that opens the register bank takes the switch out of circuit entirely.

In the NORMAL position the machine is locked to 1 MHz and a write to `$D07B` is accepted and discarded. In TURBO the switch accelerates nothing by itself — software still has to ask.

Behaviours documented for the real hardware

- **Disk access always throttles to 1 MHz** regardless of the speed setting. On real hardware it is the KERNAL, not the accelerator, that performs the switch — which is why fastloaders that bypass the KERNAL must do it themselves.
- **VIC-II raster interrupt timing is unaffected** by acceleration, so games depending on cycle-exact raster code are hit-and-miss.
- **Custom fastloaders usually fail.** The documented workaround is to switch to Normal for the load and back to Turbo afterwards.

Memory

4. The 24-bit address space

The 65816 addresses 16 MB as 256 banks of 64 KB. The SuperCPU divides that space as follows.

Range	Contents
\$000000-\$00FFFF	Bank 0 — active computer RAM image; C64-visible and shadowed
\$010000-\$01FFFF	Bank 1 — writable PseudoROM/RAM image
\$020000-\$F5FFFF	User SuperRAM (SIMM), when fitted
\$F60000-\$F7FFFF	System RAM: physical SIMM banks 0 and 1, relocated and reserved
\$F80000-\$FFFFFF	SuperCPU system ROM

The SIMM's physical addressing begins at offset zero, but the SuperCPU's own SRAM already occupies logical banks \$00 and \$01. The corresponding first 128 KB of the SIMM is therefore relocated to logical banks \$F6-\$F7 and reserved. On a 16 MB system, bank \$F5 is the highest bank available to user programs.

5. Bank 0 and the C64 PLA

Within bank 0 the map follows ordinary C64 rules, driven by the *emulated* \$01 together with the real /GAME and /EXROM. With no cartridge fitted:

Region	Contents
\$A000-\$BFFF	BASIC if LORAM and HIRAM, else RAM
\$D000-\$DFFF	RAM if LORAM and HIRAM are both low; else character ROM if CHAREN low; else I/O
\$E000-\$FFFF	KERNAL if HIRAM, else RAM
everything else	RAM

6. The bank-1 ROM shadow

The accelerator keeps ROM images in its own fast SRAM — bank 1 — and serves bank-0 reads from there, so the processor never waits on the machine's slow ROM.

The direction matters and is easy to get backwards. The table reads **bank 1** → **bank 0**: the left column is where the data physically lives in the accelerator's SRAM, the middle column is the bank-0 address at which it appears, and the right column is the condition under which the mapping is active.

SuperCPU OS Internal Memory Mirroring				
BANK 1 MAP		BANK 0 MIRROR		
Address	Reference	Mirrored at	State	Mirrored if
\$E000-\$FFFF	KERNAL	\$E000-\$FFFF	read only	CPU H/W registers disabled and KERNAL ROM enabled
\$D400-\$DFFF	—	no mirror		
\$D200-\$D3FF	extra RAM for I/O area	\$D200-\$D3FF	read always, write if CPU H/W registers enabled	I/O enabled
\$C000-\$D1FF	—	no mirror		
\$A000-\$BFFF	BASIC	\$A000-\$BFFF	read only	BASIC ROM enabled
\$8000-\$9FFF	RLDOS	\$8000-\$9FFF	read only	CPU DOS extensions enabled
\$6000-\$7FFF	ALT. KERNAL	\$E000-\$FFFF	read only	CPU H/W registers enabled and KERNAL ROM enabled
\$1000-\$5FFF	RLDOS	\$1000-\$5FFF	read only	CPU DOS extensions enabled
\$0000-\$0FFF	—	no mirror		

Bank 1 is the accelerator's own SRAM. Bank 0 is what the program sees.

OPENING THE REGISTERS SWAPS THE KERNAL

Read the \$6000-\$7FFF row carefully. Bank-0 \$E000-\$FFFF **always** serves the KERNAL — what changes is *which image*. With the hardware registers closed it comes from bank 1 \$E000-\$FFFF ; with them open it comes from a different image held at bank 1 \$6000-\$7FFF , the "alternate KERNAL".

So \$D07E does not merely expose registers — it changes the code running at the KERNAL entry points. That is why software is told to close the bank promptly, and why it is not a safe general-purpose configuration wrapper.

Hardware vector mapping

Where the processor fetches its interrupt vectors from is a three-way decision, not a two-way one.

SuperCPU Hardware Vector Mapping	
Vector address	Condition
\$00FFE4-\$00FFFF	KERNAL ROM switched out
CPU ROM	65816 in native mode
	CPU H/W registers enabled
	RAMLink H/W enabled
	CPU DOS extensions enabled
	System 1 MHz mode enabled
\$017FE4-\$017FFF	all other conditions

Any one of the five middle conditions routes vectors into the accelerator's own ROM.

7. The private RAM windows

Two pages inside I/O space are genuine RAM inside the cartridge, and must never reach the C64.

Range	Size	Purpose
\$D200-\$D2FF	256 B	SuperCPU DOS / system scratch
\$D300-\$D3FF	256 B	Free for SuperCPU-aware programs

Both pages always read back, but writes normally take effect only while the hardware-register bank is open. `$D27E` is the sole exception — writable with the bank closed.

WHY THE WRITE GATE IS NOT COSMETIC

CMD's own DOS 2.04 startup writes `$FF` to `$D20C` with the bank closed, *expects the write to be discarded*, and later branches on reading the location back. An implementation that honours that write sends the boot down the wrong path. The discarded write is the mechanism.

Registers

Almost every SuperCPU register is a write-sensitive switch: storing any value triggers the action and the value is discarded. This is why BASIC toggles speed with `POKE 53370,0` — the zero carries no meaning.

8. Write-sensitive switches, \$D07x

Optimization mode select

Declares which region the VIC-II actually reads, so the accelerator can skip keeping the rest coherent. Requires the register bank to be open.

Address	Decimal	Function
\$D074	53364	VIC bank 2, \$8000-\$BFFF — documented as "GEOS optimization"
\$D075	53365	VIC bank 1, \$4000-\$7FFF
\$D076	53366	BASIC — mirror only \$0400-\$07FF
\$D077	53367	No optimization — mirror everything (V1 default)
\$D078	53368	SIMM configuration — takes a value, not a switch
\$D079	53369	Mirror of \$D07B

There are four optimization selects, not six. The mode lives in bits 7-6 of an internal register, taken from the low two bits of the address: `$D074` → 00, `$D075` → 01, `$D076` → 10, `$D077` → 11. That register is readable and writable at `$D0B3` (v2) and `$D0B4`, and its low three bits are OR'd into **every** read of the `$D0Bx` block, whichever address was used.

Speed select

Always available, open bank or not — which is what makes `POKE 53370,0` usable straight from BASIC.

Address	Decimal	Function
\$D072	53362	system 1 MHz enable
\$D073	53363	system 1 MHz disable
\$D07A	53370	software 1 MHz enable — Normal (1 MHz, or 2 MHz in 128 Fast mode)
\$D079, \$D07B	53369, 53371	software 1 MHz disable — Turbo

Register bank enable

Address	Decimal	Function
\$D07E	53374	Enable hardware registers
\$D07F	53375	Disable hardware registers, restoring the stock map

9. The status block, \$D0B0-\$D0BF

This is *not* a mirrored block — each address carries distinct flags, and several are writable.

Address	Dec	Contents
\$D0B0	53424	version and mode, bits 7-6
\$D0B2	53426	b7 hardware registers enabled · b6 system at 1 MHz
\$D0B3	53427	v2 enhanced optimization; readable always, writable while the bank is open
\$D0B4	53428	bits 7-6 current optimization mode
\$D0B5	53429	b7 JiffyDOS switch · b6 speed switch (1 = Normal)
\$D0B6	53430	b7 processor emulation mode · b6 reset switch (v1 only)
\$D0B8	53432	b7 software speed flag · b6 master speed flag (1 = Normal)
\$D0BC	53436	b7 DOS extension mode · b6 RAMLink registers

\$D0B0 bits 7-6	Meaning
00xxxxxx	V2 in C128 mode
01xxxxxx	V2 in C64 mode
11xxxxxx	V1, no SuperCPU, or disabled

10. Detecting the accelerator

Two independent methods, both reliable.

Register test

Read \$D0BC (53436) and test bit 7. It reads **1** on a stock C64 or C128, because nothing decodes the address and an unused I/O read floats high, and **0** on a SuperCPU. Then read \$D0B0 for the version.

Detection is independent of speed — simply having the accelerator turned on is all that is required.

Processor test

The NMOS 6502/6510 computes the N flag incorrectly after a decimal-mode add. The 65816 does not.

```
SED
LDA #$99
CLC
ADC #$01      ; N set   -> 6510 is running
              ; N clear -> 65816 is running
CLD
```

Speed test

Bit 6 of \$D0B8 reads 1 in 1 MHz mode and 0 in turbo.

Programming the 65816

11. Processor modes

The E flag selects **emulation mode** (6502-compatible, 8-bit, 64 KB) or **native mode**. It is not directly accessible; it swaps with the carry flag.

```
CLC
XCE          ; -> native mode
SEC
XCE          ; -> emulation mode
```

In native mode two new status bits set register width:

Bit	Flag	Meaning
5	M	0 = 16-bit accumulator and memory · 1 = 8-bit
4	X	0 = 16-bit index registers · 1 = 8-bit

```
REP #$20      ; 16-bit accumulator
SEP #$20      ; 8-bit accumulator
REP #$10      ; 16-bit X and Y
SEP #$10      ; 8-bit X and Y
REP #$30      ; both 16-bit
```

Width is a per-region decision, not a permanent one. Sixteen-bit arithmetic can halve the instruction count — a 16-bit subtraction takes three steps where an 8-bit one takes six — but best performance comes from switching where it pays rather than staying wide throughout.

X TRANSITIONING FROM 0 TO 1

Setting the X flag to select 8-bit index registers *zeroes the high bytes of X and Y*. They are not preserved and cannot be recovered by widening again.

12. Registers the 6502 did not have

Register	Purpose	Access
B	Hidden high half of the accumulator	XBA swaps A and B
D	Direct Page — relocates "zero page" anywhere in bank 0	TCD, TDC, PHD, PLD
DBR	Data Bank — supplies the bank byte for 16-bit addresses	PHB, PLB
PBR	Program Bank — where code is executing	set by long jumps
S	16-bit stack pointer; the stack can live anywhere in bank 0	TCS, TSC, TXS, TSX

There is no direct load for the data bank, so it is set through the stack:

```
LDA #$03
PHA
PLB          ; DBR = 3, so INC $2000 now touches $032000
```

13. Addressing modes and instructions

New addressing modes

Direct page indirect — LDA (\$FD)

No Y register needed. Removes the standard LDY #\$00 : LDA (\$FD),Y idiom and the need to preserve Y.

Long — LDA \$058100

Full 24-bit addressing with no mode switch required. Native mode is *not* needed to use long addressing.

Long indirect — LDA [\$FC],Y

A **three-byte** pointer: low, high, bank. Reaches all 16 MB through a direct-page pointer.

Absolute indexed indirect jump — JMP (\$7000,X)

Also JSR (\$7000,X) . Makes a jump table trivial; entries are two bytes, so the index advances by two.

Stack relative — LDA 3,S

The stack pointer is the base of an array. Indices start at 1, because S points at the next free byte.

Stack relative indirect indexed — LDA (3,S),Y

Dereferences a pointer that was passed on the stack.

Not every instruction takes the long forms: LDA and STA do; INC and ROL do not.

New instructions worth knowing

Instruction	Effect
BRA	Branch always, ± 127 bytes, no flag needed
BRL	Branch long, 3 bytes, full 64 KB range, still position-independent
STZ	Store zero without touching the accumulator — abs, dp and X-indexed forms only, no Y-indexed form
TSB / TRB	Test-and-set / test-and-reset bits in memory; only Z is affected
PEA	Push a 16-bit constant, always 16-bit regardless of M
PEI	Push a 16-bit value read from a direct-page location
PER	Push a PC-relative address — the basis of position-independent code
PHX/PLX, PHY/PLY	Index registers straight to and from the stack
MVN / MVP	Block move — see §15
JSL / RTL	Long call and return across banks
XBA	Swap A and B

PHP is always 8 bits. Sixteen-bit pushes go **high byte first**.

14. Interrupts

Vector addresses differ by mode.

Vector	Emulation	Native
IRQ	\$FFFE/FF	\$FFEE/EF
NMI	\$FFFA/FB	\$FFEA/EB
BRK	via IRQ	\$FFE6/E7
COP	\$FFF4/F5	\$FFE4/E5
ABORT	\$FFF8/F9	\$FFE8/E9
RESET	\$FFFC/FD	—

THE KERNAL INDIRECTION IS BYPASSED

In native mode the hardware IRQ vector no longer points at the KERNAL dispatcher, so **\$0314/\$0315 is not consulted**. A native-mode program owns the interrupt outright.

Save the status register

An interrupt can arrive with M and X in any state. A native-mode handler must `PHP` on entry and `PLP` before its pulls, or the `PLA` / `PLX` / `PLY` will restore the wrong number of bytes.

The \$D019 rule

Acknowledge a VIC-II raster interrupt with a load followed by a store:

```
LDA $D019
STA $D019
```

WHY DEC \$D019 IS NOT EQUIVALENT

`DEC $D019`, `ASL $D019` and friends **work in emulation mode but fail in native mode**.

The reason is the read-modify-write cycle pattern. The NMOS part performs one read and two writes: the *original* value goes back to the bus first, and because that value has the relevant bit set, the write itself clears the latch. The CMOS core replaces that first write with an internal cycle in native mode, so nothing acknowledges the interrupt and a raster handler re-enters forever.

The complete behaviour has three cases, not two:

Mode	Cycle pattern
E = 1	read, write <i>original</i> , write new — the acknowledge works
E = 0, m = 1	read, read, write — an internal cycle instead of the first write
E = 0, m = 0	the two writes go high byte first , the reverse of the reads

What the speed is actually for

At 20 MHz a raster handler no longer has to busy-wait for the beam. Update the VIC registers and return; the recovered time is roughly 80% of what a wait-loop handler burns.

15. Block moves

`MVN` and `MVP` move up to 64 KB between any two banks at about seven cycles per byte. Set the accumulator to **count minus one**, X to the source address and Y to the destination.

```
MVN $target_bank, $source_bank
```

C = 0 moves one byte; C = \$FFFF moves 65,536. Four traps attend it.

FOUR TRAPS

1. **The operand order in the object code is destination bank, then source bank** — the reverse of how the mnemonic reads.
2. **The direction is the opposite of WDC's names.** `$54`, "Move Negative", increments. Go by increment/decrement and ignore both names.
3. **DBR is destroyed** — it is set to the destination bank. Bracket the move with `PHB` / `PLB`.
4. **It is interruptible.** One byte executes per pass with the PC parked on the opcode, so an interrupt taken mid-move pushes the address of the `MVN` itself and `RTI` resumes from X, Y and C. A handler must therefore preserve X, Y *and* the accumulator, or the move resumes corrupted.

With 8-bit index registers the move is confined to page 0 of each bank, because XH and YH are architecturally zero. Practically useless, but that is what it does.

16. Running code outside bank 0

`JSL` / `RTL` and long `JMP` update the Program Bank register. The stack, zero page and direct page stay in bank 0 regardless.

KERNAL ROUTINES ONLY WORK FROM BANK 0

They return with a two-byte `RTS`, which cannot carry a bank. And hardware stays where it is: the SID is always at \$00D400, so a player running from bank 2 still has to write its registers in bank 0.

17. Address wrapping — the complete rules

Wrapping is where the 65816 most often surprises a 6502 programmer, and it is not one rule but several.

Mode	Behaviour
Direct page, dp	16-bit wrap within bank 0
Direct page indexed, dp, X	Wraps at \$FF — 6502 style — only in emulation mode with an aligned direct page (D low byte zero). Otherwise 16-bit.
Absolute indexed, abs, X	24-bit. Carries into the next bank. This is the one place emulation mode is not a 6502.
Long indexed, long, X	24-bit, no wrap at the bank boundary
Stack, native	Full 16-bit pointer, always in bank 0
Stack, emulation	Two behaviours — see below

THE CLASSIC \$FFFF BUG

A single loop that initialises the NMI/IRQ vectors and then runs on into zero page:

```

loop    LDX #$00
        LDA data,X
        STA $FFFA,X
        INX
        CPX #$2A
        BNE loop

```

On a 6510 the store wraps to \$0000 and the zero-page half of the table lands where the programmer expected. On a 65816 the index is a 24-bit computation, so it carries into **bank 1** — \$010000, which on a SuperCPU is the SRAM holding the ROM images. Zero page is never written and the program fails later, somewhere unrelated.

The fix is two separate loops.

The emulation-mode stack split

Emulation mode has two stack behaviours, and which applies is a property of the *instruction*, not of the access. WDC names the "new" set explicitly: JSL, JSR (a,X), PEA, PEI, PER, PHD, PLD, RTL. Everything a 65C02 had — every PH*/PL*, JSR abs, RTS, RTI, BRK — plus COP and all hardware interrupts is "old".

old

Confined to page 1: only SL moves, so PHA at S=\$0100 writes \$000100 and leaves S=\$01FF.

new

A full 16-bit pointer for the duration: PEA at S=\$0100 writes \$000100 then \$0000FF, running off the bottom of the page.

Either way S is forced back inside page 1 at the end of the instruction, which is why the final value is identical for both classes.

Decimal mode

The 65816's decimal mode is the 65C02's, not the NMOS 6502's: **N and Z are computed correctly** from the decimal result. That difference is what makes the processor-detection routine in §10 work. Unlike the 65C02, there is no extra cycle in decimal mode.

Writing It in Assembly

Part IV described what the processor does. This part is about getting an assembler to emit it — which turns out to be where most of the difficulty actually lives.

The notation below follows the WDC 65c816 *Assembly Language Standard*, which is the closest thing the architecture has to a reference syntax. Real assemblers vary: ca65, ACME, 64tass, Merlin and DASM all spell some of this differently, and some do not implement the typed-equate system at all. Check your assembler's manual for the exact spelling — but the *problems* the syntax exists to solve are the same in every one of them.

18. Addressing-mode notation

Twenty-four addressing modes is a lot to hold in your head, and several of them look almost identical on the page. The distinguishing marks are the bracket style — round for a 16-bit pointer, square for a 24-bit one — and the suffix.

Mode	Written as	Effective address
Immediate	LDA #\$12	the operand itself; width follows M or X
Direct page	LDA \$12	D + operand, bank 0
Direct page indexed	LDA \$12,X	D + operand + X, bank 0
Absolute	LDA \$1234	DBR : operand
Absolute indexed	LDA \$1234,X	DBR : operand + X — <i>carries into the next bank</i>
Long	LDA \$123456	operand, all 24 bits; DBR ignored
Long indexed	LDA \$123456,X	operand + X, all 24 bits
DP indirect	LDA (\$12)	16-bit pointer at D+\$12, then DBR
DP indirect indexed	LDA (\$12),Y	as above, then + Y
DP indexed indirect	LDA (\$12,X)	pointer at D+\$12+X, then DBR
DP indirect <i>long</i>	LDA [\$12]	24-bit pointer at D+\$12
DP indirect long indexed	LDA [\$12],Y	as above, then + Y
Stack relative	LDA 3,S	S + operand, bank 0
Stack relative indirect indexed	LDA (3,S),Y	pointer at S+3, then DBR, then + Y
Absolute indirect	JMP (\$1234)	pointer read from bank 0
Absolute indexed indirect	JMP (\$1234,X)	pointer at operand + X, in the program bank
Block move	MVN \$01,\$02	destination bank, source bank

THE LEADING PARENTHESIS IS NOT ARITHMETIC

An operand that *begins* with (means indirect addressing. So

```
LDA (BASE+2)      ; indirect through the pointer at BASE+2
LDA BASE+2        ; direct load from BASE+2
```

are entirely different instructions. If you need to parenthesise arithmetic at the start of an operand, restructure the expression so it does not lead with a bracket — there is no way to disambiguate it otherwise.

19. The width problem

This is the defining difficulty of 65816 assembly, it has no equivalent on the 6502, and it accounts for a large share of the bugs a newcomer will write.

The length of an immediate operand depends on the M and X flags, and those flags are runtime state. The assembler is not running your program. It cannot know what `REP` and `SEP` did, so it has to be *told*, and it emits code on the strength of what you told it.

```
SEP #$20          ; 8-bit accumulator at runtime
LONGA OFF         ; ...and tell the assembler
LDA #$FF          ; two bytes: A9 FF

REP #$20          ; 16-bit accumulator at runtime
LONGA ON         ; ...and tell the assembler
LDA #$FFFF        ; three bytes: A9 FF FF
```

The declarations come in a set — `LONGA` and `SHORTA` for the accumulator, `LONGI` and `SHORTI` for the index registers — and underneath they are shorthand for the assembler's `ASSUME` mechanism, which also tracks the assumed data bank and direct page.

WHAT GOES WRONG WHEN THEY DISAGREE

Suppose you write `REP #$20` but forget the `LONGA ON`. The assembler still believes the accumulator is 8-bit, so it emits `A9 FF` — two bytes. The processor, which really is in 16-bit mode, reads a **three**-byte instruction: it takes `FF` as the low half of the operand and then eats the first byte of whatever instruction you wrote next as the high half.

Execution continues from the middle of that instruction. There is no diagnostic, no crash at the point of the mistake, and the symptom appears somewhere unrelated. This is the single most common way 65816 code fails.

Making it survivable

Three habits remove most of the risk.

1. **Change the flag and the declaration on adjacent lines, always.** Never let a `REP` or `SEP` appear without its declaration immediately below it. Treat them as one two-line instruction.
2. **Declare the width at the top of every subroutine**, and document which widths the routine expects on entry. A subroutine that is correct only when the caller happens to be in 8-bit mode is a latent bug, not a working routine.
3. **Force a known state at boundaries** — interrupt handlers, callbacks, anything the caller does not control. `PHP / REP -or- SEP / work / PLP` costs a handful of cycles and removes an entire class of failure.

Note that `REP` and `SEP` take a *mask*, not a flag number, and the mask bits are the P-register bits. `#$20` is M, `#$10` is X, `#$30` is both. They also reach the ordinary flags — `SEP #$08` sets decimal mode just as `SED` does.

20. Forcing the addressing mode

The assembler picks the shortest encoding that can reach the address. That is usually what you want and occasionally catastrophic, because two of the three inputs to the decision are runtime registers that the assembler can only assume.

Direct page vs absolute

Whether `$12` is reachable as a direct-page address depends on `D`. The assembler works from an assumed value; if `D` is something else at runtime, the access lands in the wrong place.

Absolute vs long

Whether a 16-bit address reaches your data depends on `DBR`. Same problem.

The standard provides explicit suffixes that override the choice:

Suffix	Forces	Example and encoding
<code>:D</code>	direct page, 1-byte operand	<code>LDA \$10:D → A5 10</code>
<code>:A</code>	absolute, 2-byte operand	<code>LDA \$0010:A → AD 10 00</code>
<code>:L</code>	long, 3-byte operand	<code>LDA \$000010:L → AF 10 00 00</code>
<code>:S</code>	short (relative) branch	forces BRA-style over a long branch

The complementary half of the mechanism is that **symbols carry a type**. An address defined with a direct-page equate is a direct-page address everywhere it is used, and the assembler picks the encoding from the symbol rather than from the numeric value. This is why the standard has three separate equate directives instead of one, and it is worth using them properly rather than defining everything as a plain number.

21. Directives and program structure

Typed equates

Directive	Defines
<code>.EDP</code>	a direct-page address — 8-bit
<code>.EQU</code>	an absolute address — 16-bit
<code>.EQL</code>	a long address — 24-bit
<code>.CON</code>	a constant, not an address — 32-bit

The distinction between an *address* and a *constant* is enforced, which catches a whole family of mistakes where a number is used as a pointer or vice versa.

Data

Directive	Emits
.BYTE	1 byte per operand
.WORD	2 bytes per operand
.LONG	4 bytes per operand
.OFFS	2 bytes — a 16-bit address, for jump tables within a bank
.ADRS	3 bytes — a 24-bit address, for long pointers

`.ADRS` is the one to reach for when building a table of long pointers for `[dp],Y` to walk, since it lays down exactly the three bytes that mode expects.

Repetition uses a `DUP` count, which nests, so a table of structures can be declared without writing it out:

```
BUFFER .BYTE 256 DUP (0) ; 256 zero bytes
GRID .WORD 8 DUP (4 DUP ($FFFF)) ; 32 words of $FFFF
```

Procedures and the RTS/RTL problem

The 65816 has two returns, and picking the wrong one corrupts the stack and jumps somewhere arbitrary. `RTS` pops a 16-bit address; `RTL` pops 24 bits. A routine entered with `JSR` must end with `RTS`; one entered with `JSL` must end with `RTL`.

The standard's answer is to declare the procedure's kind and let the assembler choose:

```
DrawLine .PROC far
...
RET ; assembles as RTL, because the proc is far
.ENDP
```

Labels inside a `.PROC` are local to it unless exported with `.GLOBAL`, which is what makes short scratch names like `loop` and `done` reusable across a large source file.

Local labels

Numeric local labels avoid inventing names for the twentieth loop in a file. Declare with `^0` through `^9` and refer backwards with `<n` or forwards with `>n`:

```
^1 LDA (SRC),Y
STA (DST),Y
DEY
BNE <1 ; branch back to the nearest ^1
```

Segments

Code, data, direct page, stack and constants are declared as separate segments with `.SEGMENT` / `.ENDS`, each with its own alignment and origin. On a machine where the direct page can be moved anywhere and the stack can be any size, being able to declare "this block belongs in direct page" and let the linker place it is worth more than it is on a 6502.

22. Idioms and traps

A 16-bit counted loop

With a 16-bit accumulator the counter no longer has to live in memory:

```
REP #$30                ; 16-bit A, X and Y
LONGA ON
LONGI ON
LDX #0
LDY #COUNT-1
^1 LDA TABLE,X
  STA DEST,X
  INX
  INX                ; two, because entries are words
  DEY
  BPL <1
```

Relocating the direct page

Direct-page addressing is a byte shorter and a cycle faster than absolute. On a 6502 that speed is confined to page zero, which the KERNAL has already largely claimed. On the 65816 you can simply move it:

```
REP #$20
LONGA ON
LDA #WORKAREA           ; any 16-bit address in bank 0
TCD                     ; direct page now starts there
```

TWO CONSEQUENCES OF MOVING D

Every existing direct-page reference in scope now points somewhere else — including any KERNAL call you make while **D** is moved. Restore it before calling code you did not write.

And if **D** is not page-aligned, emulation-mode **dp,X** stops wrapping at **\$FF** and becomes a 16-bit computation. See §17.

Passing parameters on the stack

PEA pushes a constant without disturbing a register, and stack-relative addressing reads it back, so a call can carry arguments without reserving globals:

```
PEA #WIDTH              ; push arguments
PEA #HEIGHT
JSR DrawBox
PLA                     ; caller cleans up
PLA
...
DrawBox LDA 5,S          ; height (3,S is the return address)
      LDA 7,S          ; width
      RTS
```

Count the offsets carefully: the return address **JSR** pushed sits between the stack pointer and your arguments, and **JSL** pushes three bytes rather than two, which shifts every offset by one.

Traps worth a checklist

1. **Width declaration out of step with the runtime flags** — §19. If code that assembled cleanly runs off into nonsense, suspect this before anything else.
2. **RTS from a routine entered with JSL**, or the reverse. Declare procedures near or far and use `RET`.
3. **Setting X to 8 bits destroys the high bytes of both index registers.** They are zeroed, not stashed.
4. **MVN leaves DBR set to the destination bank.** Bracket it with `PHB` / `PLB`.
5. **An interrupt can arrive in any width.** A handler that does not `PHP` on entry and `PLP` before its pulls will restore the wrong number of bytes — §14.
6. **Stack-relative offsets shift when the call type changes.** Converting a `JSR` routine to `JSL` silently moves every argument by one byte.
7. **Long addressing does not need native mode**, but 16-bit registers do. Reaching bank 2 from emulation mode is perfectly legal.

23. Worked examples

Written for **64tass 1.6**. Its width directives map directly onto the problem in §19: `.as` / `.al` declare the accumulator 8- or 16-bit and `.xs` / `.xl` do the same for the index registers. 64tass also offers `.autsiz`, which tracks `rep` / `sep` for you — worth turning on, though the explicit form is used below so the state stays visible on the page.

Detecting the accelerator

```
.cpu "65816"
*= $c000

start  lda $d0bc                ; bit 7: 0 = SuperCPU, 1 = stock machine
      bmi no_scpu

      lda $d0b0                ; version / mode in bits 7-6
      and #$c0
      cmp #$c0
      beq is_v1                ; %11 = V1, or disabled
      cmp #$40
      beq is_v2_64             ; %01 = V2 in C64 mode
      bra is_v2_128            ; %00 = V2 in C128 mode
```

Bracketing timing-critical code

Both registers are write-sensitive, so the stored value is discarded and whatever happens to be in the accumulator is fine.

```
sta $d07a                ; -> 1 MHz (53370)
jsr fastloader            ; anything that counts cycles
sta $d07b                ; -> turbo (53371)
```

A 16-bit screen fill

```
    clc
    xce                ; native mode
    rep #$30           ; 16-bit accumulator and index
    .al
    .xl

    lda #$2020         ; two space characters per store
    ldx #$0000
-   sta $0400,x
    inx
    inx
    cpx #$03e8         ; 1000 bytes
    bcc -

    sep #$30
    .as
    .xs
    sec
    xce                ; back to emulation mode
```

Note `.al / .xl` immediately after the `rep`, and their partners after the `sep`. Omit them and `lda #$2020` assembles as two bytes while the processor reads three — \$19.

Reaching SuperRAM directly

Long addressing needs no mode switch; 16-bit registers do.

```
    lda $028000        ; read bank $02 directly
    sta $030000        ; write bank $03

    ; a 24-bit pointer in direct page, walked with Y
    lda #$00
    sta $fc            ; low
    sta $fd            ; high
    lda #$02
    sta $fe            ; bank
    ldy #$05
    lda [$fc],y        ; = $020005
```

Loading a file into SuperRAM from an IEC drive

This is the pattern most SuperRAM software needs, and four constraints apply at once. The KERNAL is 6502 code, so it must be called in **emulation mode with 8-bit registers**; it returns with `rts`, so it must be called **from bank 0**; its addresses are 16-bit, so it **cannot load into bank 2** at all; and disk timing breaks at 20 MHz, so the transfer runs at **1 MHz**.

The way through is a staging buffer in bank 0 and a block move per chunk.

```

buffer = $c000          ; 4K staging area, bank 0
bufsize = $1000
dstbank = $02           ; first SuperRAM bank

    .cpu "65816"
    *= $0810

load    lda $d0bc        ; refuse politely on a stock machine
        bmi no_scpu
        sta $d07a        ; 1 MHz for the whole transfer

        lda #1           ; logical file 1, device 8, secondary 0
        ldx #8
        ldy #0
        jsr $ffba        ; SETLFS
        lda #namelen
        ldx #<name
        ldy #>name
        jsr $ffbd        ; SETNAM
        jsr $ffc0        ; OPEN
        bcs io_error
        ldx #1
        jsr $ffc6        ; CHKIN
        bcs io_error

        stz dstoff       ; destination offset within dstbank
        stz dstoff+1
        stz pages

fill    jsr $ffb7        ; READST
        bne done         ; nonzero = EOF or error
        jsr $ffcf        ; CHRIN
        ldx count
        sta buffer,x
        inc count
        bne fill         ; 256 bytes gathered when it wraps
        inc pages
        lda pages
        cmp #bufsize/256
        bne fill

        jsr stage_to_bank ; buffer is full, move it out
        stz pages
        bra fill

done    jsr stage_to_bank ; move the partial tail
        jsr $ffcc        ; CLRCHN
        lda #1
        jsr $ffc3        ; CLOSE
        sta $d07b        ; turbo again
        rts

```

Only the move needs native mode, and it is entered and left around each chunk so every KERNAL call still happens in emulation mode with 8-bit registers:

```

stage_to_bank
    clc
    xce                    ; native
    rep #$30
    .al
    .xl

    phb                    ; MVN destroys the data bank
    lda #bufsize-1         ; count MINUS ONE
    ldx #buffer            ; source offset, bank $00
    ldy dstoff             ; destination offset, bank $02
    mvn $00,dstbank        ; operand order: verify it, see below
    plb

    lda dstoff             ; advance the destination
    clc
    adc #bufsize
    sta dstoff

    sep #$30
    .as
    .xs
    sec
    xce                    ; back to emulation for the next KERNAL call
    rts

```

VERIFY THE MVN OPERAND ORDER BEFORE TRUSTING IT

The object code always carries **destination bank first, then source bank**, but assemblers disagree about which order the *mnemonic* takes — and getting it backwards silently moves the wrong data over something you cared about.

Do not take the line above on faith, and do not take it from a manual either. Assemble one move on its own, disassemble it, and read the bytes:

```

    mvn $00,$02           ; assemble this alone
                          ; 54 xx yy  ->  xx is the DESTINATION bank

```

Thirty seconds, once per assembler, and the question never comes back.

WHY THE STAGING BUFFER IS UNAVOIDABLE

No KERNAL entry point loads into a bank other than 0. `SETLFS` and its relatives take 16-bit addresses and the routines return with `rts`, so even calling them from bank 2 would fail. Anything that fills SuperRAM from disk stages through bank 0 and moves the data itself.

The move is nearly free next to the transfer: `MVN` runs at about seven cycles per byte against a drive delivering a few hundred bytes per second.

A native-mode raster handler

```
irq      php                      ; the interrupted code's widths are unknown
        rep #$30
        .al
        .xl
        pha
        phx
        phy

        lda $d019                ; acknowledge -- NOT dec/asl, see §14
        sta $d019

        ; ... handler body ...

        ply
        plx
        pla
        plp                      ; restores M and X before the rti
        rti
```

Pull in reverse order and leave `plp` last: until it executes the pulls are still 16-bit, which is what makes them match the pushes. In native mode the hardware vector is `$FFEE/$FFEF` and the KERNAL's `$0314` indirection is not consulted — §14, and Part II for where that vector is actually fetched from.

Driving It from BASIC

Every speed and status register is reachable with PEEK and POKE, which makes BASIC a perfectly good place to detect the accelerator, control it and measure it — right up to the point where you want the extra memory, which it cannot reach at all.

24. Detection and status

The same bit as §10, from BASIC:

```
10 IF (PEEK(53436) AND 128) = 0 THEN 100
20 PRINT "NO SUPERCPU FITTED" : END
100 PRINT "SUPERCPU PRESENT"
```

A fuller readout. Everything here is read-only and safe: nothing is POKEd, and none of it needs the hardware-register bank opened.

```
100 V = PEEK(53424) AND 192           : REM $D0B0 VERSION / MODE
110 IF V = 0 THEN PRINT "V2, C128 MODE"
120 IF V = 64 THEN PRINT "V2, C64 MODE"
130 IF V = 192 THEN PRINT "V1, OR DISABLED"
140 :
150 S = PEEK(53432)                   : REM $D0B8 SPEED FLAGS
160 IF (S AND 64) THEN PRINT "RUNNING: 1 MHZ" : GOTO 180
170 PRINT "RUNNING: TURBO"
180 :
190 W = PEEK(53429)                   : REM $D0B5 PHYSICAL SWITCHES
200 IF (W AND 64) THEN PRINT "SWITCH: NORMAL" : GOTO 220
210 PRINT "SWITCH: TURBO"
220 IF (W AND 128) THEN PRINT "JIFFYDOS: ON" : GOTO 240
230 PRINT "JIFFYDOS: OFF"
240 END
```

Remember from §3 that the switch position and the running speed are different questions. NORMAL locks the machine to 1 MHz; TURBO only *permits* acceleration, and software still has to ask for it.

25. Speed control, and measuring it

```
POKE 53370,0 : REM $D07A -> 1 MHZ
POKE 53371,0 : REM $D07B -> TURBO
```

The zero carries no meaning; these are write-sensitive switches and the value is discarded. Both work without opening the register bank, which is exactly what makes them usable from BASIC.

An anchored benchmark

A speed figure with nothing to compare against says very little. This measures both states in one run, so the ratio anchors itself:

```

10 IF (PEEK(53436) AND 128) THEN PRINT "NO SUPERCPU" : END
20 POKE 53371,0 : REM TURBO
30 TI$ = "000000"
40 FOR I = 1 TO 10000 : NEXT I
50 T1 = TI
60 POKE 53370,0 : REM 1 MHZ
70 TI$ = "000000"
80 FOR I = 1 TO 10000 : NEXT I
90 T2 = TI
100 POKE 53371,0 : REM BACK TO TURBO
110 PRINT "TURBO :"; T1; "JIFFIES"
120 PRINT "NORMAL:"; T2; "JIFFIES"
130 PRINT "SPEEDUP:"; T2 / T1

```

TI counts jiffies — sixtieths of a second — and it is driven by the CIA rather than by the processor, so it keeps real time in both states. That is what makes it a valid clock for this measurement.

Expect the ratio to fall well short of twenty. BASIC spends much of its time in memory the accelerator serves at full speed, but every screen and I/O touch still costs a real 1 MHz bus cycle, and an empty **FOR/NEXT** is close to the best case. This is the honest number to quote, and more useful than the headline clock ratio.

26. What BASIC cannot do

PEEK AND POKE CANNOT REACH SUPERRAM

They take 16-bit addresses, so everything from bank **\$02** upwards is unreachable from BASIC. Using the expansion memory requires 65816 code — long addressing or a block move — with BASIC reduced to **SYS** ing a machine-language helper. CMD said as much when they shipped the card: SuperRAM does not make existing BASIC programs use the extra memory.

DO NOT OPEN THE REGISTER BANK CASUALLY FROM BASIC

POKE 53374,0 is tempting as a wrapper for the optimization and private-RAM registers, which do need the bank open. Resist it.

\$D07E also **changes which KERNAL image is mapped**: the alternate image held at bank 1 **\$6000-\$7FFF** takes over at **\$E000-\$FFFF**, as shown in §6 — and BASIC calls into the KERNAL constantly. Opening the bank from the READY prompt swaps the code out from underneath the interpreter. Anything that opens it should close it in the same machine-language routine, promptly, with no KERNAL calls in between.

Reads of **\$D200-\$D3FF** work with the bank closed, so inspecting the private RAM from BASIC is safe. Only writes need the bank, and they belong in machine code.

Compatibility

Software that fails on an accelerated machine mostly fails for one of nine reasons, and all nine are properties of the real hardware rather than of any particular emulation.

27. The nine rules

1 — No illegal opcodes

The 6510's undocumented opcodes save a cycle or two by fusing operations. The 65816 defines **every** value from `$00` to `$FF` as a real instruction, so those byte values now mean something entirely different and the program crashes rather than misbehaving quietly.

2 — Speed control belongs to the program

```
STA $D07A    ; 53370 – drop to 1 MHz
STA $D07B    ; 53371 – return to turbo
```

Both are write-sensitive; the value is discarded.

3 — Disk access must run at 1 MHz

Fast loaders and JiffyDOS encode data in the *timing* between edges, so they break at 20 MHz. On real hardware it is the KERNAL, not the accelerator hardware, that drops the speed when its own disk routines are used — so a program with its own IRQ loader or fastloader has to do it by hand.

4 — Timing-critical code must run at 1 MHz

The same applies to anything built on 1 MHz cycle counting: FLI, side-border opening, raster bars. Drop to 1 MHz for the effect, return to turbo for the work. Routines written *for* 20 MHz can of course do far more per frame.

5 — The address space does not wrap at `$FFFF`

See §17.

6 — There is no 2 MHz

Some C64 programs poke `$D030` for 2 MHz on a C128 in C64 mode with the screen blanked. That gains nothing with a SuperCPU fitted; offer 20 MHz instead.

7 — Only documented I/O locations

The C64 decodes the VIC incompletely, so `$D220` behaves like `$D020` and `$D212` like `$D012`. Programs relying on this break on a SuperCPU, which does not decode the mirrors — and, worse, uses that space itself. `$D220` falls inside the `$D200-$D2FF` private RAM window.

8 — Ultimex cartridges

See Part I. A freezer that replaces the KERNAL leaves the accelerator inoperable.

9 — Detection before divergence

Detect with `$D0BC` bit 7 and select alternative routines, rather than assuming either machine.

28. Why acceleration sometimes buys nothing

DO NOT CALCULATE INSIDE THE IRQ

Frame-driven code gains nothing from an accelerator. A heavier calculation simply finishes earlier, and the CPU then idles until the next raster interrupt arrives on its own 1 MHz schedule.

Work belongs in the main loop, running as far as it can get, rather than being sliced across frames. This is the single most common reason a program "does not get faster with a SuperCPU".

The corollary is a useful test of any accelerator, real or emulated: main-loop-bound code should scale with the selected speed, and IRQ-bound code should not.

29. Repairing existing software

Software that fails on an accelerated machine tends to fail in a handful of recurring shapes rather than in a thousand individual ways. That makes most of it diagnosable by symptom, and a good deal of it repairable with a very small patch — sometimes a single byte.

What follows is a catalogue of those shapes. The right of the table is where to look; the fix usually follows once the shape is identified.

Symptom	What it means	Repair
Crashes immediately, often in the protection check	Undocumented 6510 opcodes, which are real 65816 instructions with different meanings	Replace with documented equivalents, or bypass the check entirely
Countdowns finish instantly; "press fire" prompts flash past; pauses vanish	Software delay loops with no real time base — they counted cycles	Drop to 1 MHz around them, or rewrite against the raster or a CIA timer
Plays unwatchably fast, though otherwise correct	The main loop is not frame-locked; it simply ran as fast as the CPU allowed	Make the loop wait on a flag the raster interrupt sets once per frame
Flickering or torn double buffering	The main loop no longer waits for the interrupt's buffer swap, so it draws into the buffer being displayed	Have the loop wait on the swap flag before drawing
Movement far too fast even after the loop is frame-locked	Velocities and accelerations were tuned per <i>iteration</i> , and there are now many more iterations per frame	Rescale the constants once the frame lock is in place
Loading fails or hangs	A software fastloader; these break at 20 MHz essentially without exception	Drop to 1 MHz for the load, or replace it with KERNAL/JiffyDOS calls
Slowdown, flicker or lockups specifically on a V2	Zero page and stack are not being mirrored, and the program depends on the VIC seeing them	Turn that mirroring back on — the Z control in the enhanced optimization register, \$32
Hard lockup on a 128D	The program pokes \$D030 chasing 2 MHz	One byte — see below

The one-byte \$D030 fix

This one deserves singling out, because the repair is unusually clean and because the failure is severe: on a 128D, a SuperCPU meeting a write to \$D030 can lock the machine outright rather than merely failing to accelerate.

INC \$D030 and BIT \$D030 are both three bytes long, and BIT absolute touches nothing but the flags. So the patch is a single opcode byte:

```
EE 30 D0      INC $D030      ; locks up
2C 30 D0      BIT $D030      ; harmless read, same length
```

No relocation, no shifted branch targets, no free space required. Worth knowing as a search pattern too: EE 30 D0 is a short, distinctive byte sequence to hunt for in any program that misbehaves on a 128.

Finding the code to patch

The diagnostic method that makes all of this practical is a freezer cartridge, which gives a snapshot to disassemble rather than a black box to guess at. Beyond that:

- **Hunt for signature byte sequences.** Joystick reads at \$DC00 / \$DC01 lead to the movement code; EE 30 D0 finds the 2 MHz poke; writes to \$0314/\$0315 and \$FFFA / \$FFFE find the interrupt setup, which is usually where the frame lock belongs.
- **Fill before you run.** Flood suspect regions with a known byte, run the program, then look at what changed. That maps what the program actually uses, and just as usefully what it does not — which is where a patch can safely live.

- **Disassemble three places first:** initialisation, the main loop, and the interrupt handler. Nearly every fault in the table above lives in one of them.
- **Test every mode, not just the first screen.** A frame lock that works in the title sequence and fails in the two-player game is a common outcome, because the loop structure is often not shared.

A NOTE ON SCOPE

Patching software is something to do to your own copies, and a repaired program is generally not yours to redistribute. The techniques here are for making things you own run on hardware you own.

30. The host machine: power, cartridges and companions

Two whole classes of SuperCPU failure have nothing to do with the software being run. They are worth knowing before anyone starts debugging a program that is not actually at fault.

Power is the first thing to suspect

SCPU INITIALIZATION ERROR #6 MEANS THE SUPPLY, NOT THE ROM

The accelerator draws considerably more current than a stock machine, and it already exceeds Commodore's nominal cartridge-port specification on its own. **Initialization Error #6 is the documented symptom of an inadequate power supply** — not of a bad ROM image, a wrong DOS version, or an incompatible machine.

This is worth stating flatly because the error is easy to misattribute to whatever was changed most recently, and a supply that is merely marginal produces it intermittently.

Supplies that were considered adequate:

- The REU supply shipped with the 1764 package.
- A C128 supply, with a connector modification.
- Two C64 supplies through an adapter.

Adding a SuperRAM card raises the load further. With both fitted, the +5 V rail should sit near 5 V; CMD treated anything below **4.8 V** as evidence of an inadequate or failing supply, and undervoltage can cause incorrect operation or damage rather than a clean failure — see §25.

Cartridges that cannot coexist

The mechanism is in Part I: a freezer that pulls `/GAME` low with `/EXROM` high replaces the KERNAL and leaves the accelerator inoperable. Named as incompatible:

Incompatible	Compatible
Action Replay	CMD expansions, including RAMLink
Super Snapshot	SwiftLink and Turbo232
Final Cartridge III	Commodore REUs
Nordic Power	

The irony is worth noting: the freezer cartridges are exactly the tool §29 recommends for diagnosing and patching software, and they cannot be in the machine at the same time as the accelerator. Freeze and disassemble on a stock machine; test the patch with the SuperCPU fitted.

Which accelerator in which machine

Board	Machine	Result
SuperCPU 64	C64	as designed
SuperCPU 64	C128	works, but C64 mode only
SuperCPU 128	C128	native C128 acceleration; requires the MMU adapter, §37

A V1 board cannot be upgraded to V2. The optimization logic lives in a PLD whose replacement part is a different density, so the change was not field-installable.

Two operational notes

- **Speed can be changed while a program runs.** Toggling between 1 MHz and turbo does not require a reset or a reload, which is what makes the bracketing idiom in §18 and §25 practical rather than theoretical.
- **Some software defeats reset.** A program that has disabled the reset path may need the machine powered down for a while, or the accelerator temporarily removed, rather than reset again harder.

The SuperRAM Card

31. The card and its SIMM

SuperRAM is an add-in card for the SuperCPU 64 and 128. It accepts a single 72-pin fast-page-mode SIMM and provides 1, 4, 8 or 16 MB that the 65816 addresses directly. Unlike an REU or RAMLink, it is part of the processor's address space: programs can read, write and *execute* in it.

The card carries a clock oscillator, bus driver, GAL, digital delay, CPLD, supporting passives, two SuperCPU connectors and the SIMM socket. The CPLD implements most of the memory mapping, control and refresh logic.

SIMM REQUIREMENTS ARE STRICT

Only standard **fast-page-mode** 72-pin SIMMs rated at **70 ns or faster** are supported. EDO and SDRAM modules are not compatible. Faster ratings do not improve performance, because the controller uses fixed access timing.

Capacity	Organization	Row size	Row/column address bits
1 MB	256K × 32/36	2 KB	9/9
4 MB	1M × 32/36	4 KB	10/10
8 MB	2M × 32/36	4 KB	10/10 or 11/10
16 MB	4M × 32/36	4–8 KB	11/10, 12/10 or 11/11

Installed SIMM	User expansion banks
1 MB	\$02-\$0F
4 MB	\$02-\$3F
8 MB	\$02-\$7F
16 MB	\$02-\$F5

POWER

A SuperCPU alone already exceeds Commodore's nominal cartridge-port current specification, and SuperRAM increases the load. With both installed the +5 V supply should remain near 5 V; CMD considered anything below **4.8 V** evidence of an inadequate or faulty supply. Inadequate voltage can cause incorrect operation or hardware damage.

32. Detecting and allocating expansion RAM

First verify a SuperCPU ROM with expansion support: in C64 mode, read four PETSCII bytes beginning at \$00E487 (\$00F6DD in 128 mode). Version 1.40 was the first release with SuperRAM support.

Then read the allocation pointers — note that they sit inside the \$D200-\$D2FF private RAM window.

Address	Meaning
\$00D27C	First available page
\$00D27D	Bank containing the first available page
\$00D27E	Last available page + 1
\$00D27F	Bank containing the last available page + 1

All four bytes are zero when no expansion RAM is available. They are valid in bank \$00 while I/O is mapped in; with I/O mapped out, use the corresponding locations in bank \$01 .

RESERVE PROPERLY

Applications that claim SuperRAM must **update the allocation pointers** rather than assuming all fitted memory is free — future system extensions or previously loaded programs may already have reserved part of it. Relocatable code and data are strongly recommended.

To change the pointers through bank \$00 : store any value to \$00D07E to open the register bank, update the pointers, then store any value to \$00D07F to close it again.

Speed. SIMM DRAM is slower than the SuperCPU's on-board SRAM. Place latency-critical routines in SRAM where possible. The controller is optimised for sequential access; random reads and writes incur the largest slowdown.

The SuperCPU 128

33. Version 2 architecture

Doug Cotton's contemporary article *The SuperCPU Steps Up to Version 2 (Commodore World 22)* supplies details absent from most references.

- SCPU128 v2 has **256 KB of onboard SRAM**, enough to shadow the RAM and ROM images of both a C64 and a C128. SCPU64 has only the original 128 KB pair.
- Only two 64 KB images are presented to the 65816 at once. **Bank \$00 always contains the active computer RAM image**; in C128 mode that is C128 physical RAM bank 0 or 1 according to the current selection.
- **Bank \$01 contains the writable ROM image**. It normally holds the C64 ROM image, with C128 ROM segments switched into the composite when those segments are mapped.
- The extra 128 KB unique to SCPU128 is switched into banks \$00 / \$01 as needed; it is *not* exposed as additional 65816 banks. SuperRAM consequently still starts at bank \$02 , preserving the common application memory map.

34. The enhanced optimization register

The same article supplies the complete \$D0B3 v2 encoding. Two low bits matter, conventionally named **B** and **Z**.

- B** Assigns a per-bank optimization to C128 RAM bank 0 or bank 1. Meaningful only in C128 mode on an SCPU128.
- Z** Z=0 mirrors zero page and stack (\$0000-\$01FF) to the host; Z=1 stops that mirroring for faster direct-page and stack access.

\$D0B3 pattern	Meaning
00xxx1BZ	VIC bank 0, \$0000-\$3FFF
01xxx0B0	VIC bank 1, \$4000-\$7FFF
00xxx0B0	VIC bank 2 / GEOS, \$8000-\$BFFF
01xxx1B0	VIC bank 3, \$C000-\$FFFF
10xxx0B0	BASIC, \$0400-\$07FF
11xxx00Z	No optimization, both banks \$00:0000-\$01:FFFF
11xxx1BZ	No optimization, selected C128 bank \$0000-\$FFFF
10xxx100	Full optimization — no RAM mirroring at all

- Relocating zero page or stack with the C128 MMU does **not** relocate the range controlled by Z; the optimization applies to the true physical \$0000-\$01FF .

- Writes to the enhanced and legacy optimization controls are reflected into each other for backward compatibility.
- I/O is always mirrored whenever I/O is mapped in, independently of the RAM optimization mode.

35. C128 access-class timing rules

These are access-class rules, not a blanket throttle for \$D000-\$DFFF .

- In C128 mode, reads **and** writes to \$0001 and \$FF00 wait for the next 1 MHz bus cycle.
- Reads from VDC ports \$D600 / \$D601 and MMU load registers \$FF01-\$FF04 also wait for the next 1 MHz cycle; writes to those locations use the cache.
- After a real VDC access, another VDC access is blocked during the following 1 MHz cycle so the VDC can complete the operation.
- On v2, colour RAM is treated as ordinary mirrored memory rather than slow I/O. Reads are full speed, and writes are full speed when the cache is ready.
- \$D200-\$D3FF SRAM and the \$D07x / \$D0Bx registers operate at full processor speed on v2. In C64 mode \$0001 is also full speed.

36. Startup and MMU constraints

THE C128 DOES NOT COLD-BOOT ON THE 8502

Its Z80 executes first, checks the cartridge signals, selects C128 or C64 operation through the 8722, and only then hands control to the 8502. Any accelerator — or emulation of one — cannot assume the C64 sequence of immediately fetching a 65xx reset vector.

The 8722 imposes further rules the memory layer must retain:

- \$FF00 is the always-visible configuration register; \$FF01-\$FF04 load the four preset configurations represented by \$D501-\$D504 .
- \$D506 selects common RAM. A value of \$04 means 1 KB of common RAM at the bottom of memory (\$0000-\$03FF) — which is why code that changes the selected RAM bank can safely execute from \$0300 .
- Page-zero and stack relocation are distinct from common RAM *and* from the SuperCPU's Z optimization. They must be applied in the correct order.
- The processor's built-in port at \$0000 / \$0001 is special and is not ordinary relocated RAM.

37. The MMU adapter

The SuperCPU 128 does not install on the expansion port alone. It ships with a small adapter board that goes into the C128's **8722 MMU socket**: the MMU is lifted out, the adapter takes its place, the MMU is seated on top of the adapter, and a cable runs from the adapter to the accelerator. Nothing equivalent exists for the SuperCPU 64.

Why it has to exist

The reason follows directly from the v2 architecture in §32. Bank \$00 of the 65816 always holds *the active computer RAM image*, and in C128 mode that means C128 physical RAM bank 0 or bank 1 **according to the**

current C128 selection. For the accelerator to present the right image, it has to know which bank the machine has selected, continuously and without latency.

That information is not on the expansion port. The C128's memory configuration lives in the 8722: the `$FF00` configuration register, the four preset loads at `$FF01-$FF04`, the common-RAM selection, and page-zero and stack relocation. A cartridge sees addresses and data; it does not see which of two 64 KB banks those addresses resolve to, and it cannot see a configuration change at the moment it happens.

The same argument covers the startup path in §35. The C128 does not begin a cold boot on the 8502 — the Z80 runs first, inspects the cartridge signals, selects C128 or C64 operation through the 8722, and then hands over. An accelerator that must be correct from reset needs visibility of that handoff, and the socket is where it is visible.

THE SHORT VERSION

The adapter exists because the C128's memory map is decided inside a chip the expansion port cannot see into. Anything reimplementing a SuperCPU 128 from the cartridge port alone is missing those signals, and no amount of firmware recreates them.

What has actually been measured on one

One adapter has been examined in the course of this project, and one measurement is worth recording because it is easy to over-read in both directions.

An identical firmware image was run on two different C128s: one with a CMD MMU adapter fitted (8722 seated in the adapter, **SuperCPU-side header left unconnected**) and one stock machine. On a settled static screen under sustained expansion-port DMA:

Machine	Result
C128 with the adapter fitted, header unconnected	sparse cells still changed
A different, stock C128	static screen held

WHAT THAT DOES AND DOES NOT ESTABLISH

It establishes that **DRAM refresh margin differs between the two machines**. It does **not** isolate the adapter as the cause. DRAM lot, VIC-IIe revision, board revision, bus loading and per-machine bus calibration all differ between the two boards, and the adapter is one candidate among several.

The test that would isolate it has not been done: remove the adapter from the *same* board, reseal the 8722 directly, and re-run the identical image. Until then this is a machine-to-machine difference with a plausible suspect, not a finding about adapters.

It is worth noting separately that an aftermarket board sitting in the 8722 socket with a dangling header is a configuration the C128 service schematic never described, which is relevant context for any measurement on such a machine that contradicts the documented behaviour.

Wiring: what we have, and how to derive the rest

No verified pinout is reproduced here, because we do not have one. Rather than guess at it, here is how to characterise an adapter you are holding, which takes a continuity meter and about twenty minutes.

Work with the adapter out of the machine and the 8722 lifted out of it, so nothing is powered and nothing is in parallel with what you are probing.

1. **Find the pass-throughs.** Probe each pin of the adapter's upper socket — where the MMU seats — against the corresponding pin of the plug underneath. Most will read as a short. Those pins are simply passed through and carry no information for the accelerator.
2. **Find the interceptions.** Any pin that reads *open* between socket and plug is being intercepted rather than passed on. That is the interesting list, and it is short. An intercepted pin means the adapter is inserting itself into that signal, not merely observing it.
3. **Find the taps.** Probe each header pin against every socket pin to build the tap list. A header pin that reads short to a socket pin is an observation point; one that reads open to everything is either a supply, a ground, or an output the adapter drives itself.
4. **Watch for active parts.** If a pin reads open in *both* directions and to every header pin, the adapter is buffering or switching it rather than wiring it, and continuity alone will not tell you what it does. Identify the logic on the board at that point instead.

The distinction that matters most is item 1 versus item 2. A purely passive adapter that only *taps* signals is telling the accelerator what the MMU is doing. One that *intercepts* signals is letting the accelerator change what the MMU does — a considerably stronger capability, and one that would explain why the connection is mandatory rather than optional.

That single question — taps only, or taps and intercepts — is answerable from the continuity map alone, and it is the first thing worth knowing about any adapter that turns up.

Measured Hardware Behaviour

This part records findings taken from a physical Commodore 128 under sustained expansion-port DMA. They are measurements, separated from interpretation, and several contradict the published documentation.

38. The C128 under sustained DMA

Measured machine parameters

Quantity	Measured
PHI2 period	978.6 ns
Cycles per raster line	65
Lines per frame	263
Frame rate	59.78 Hz
VIC DRAM refresh	5 rows per raster line — a full 256-row sweep takes ~3.27 ms

THE CENTRAL FINDING

Holding /DMA asserted continuously **suppresses C128 DRAM refresh**. Releasing it for a sub-cycle interval (~286 ns) at a verified PHI2 falling edge restores refresh without unparking the 8502.

Skipped, not deferred

The VIC's refresh counter **free-runs**. A row whose refresh slot falls inside a traffic window is *skipped for that entire sweep* rather than deferred within it. So an 80% duty cycle means roughly 80% of rows refreshed normally and 20% skipped — not every row refreshed at 80% rate. This is why duty arithmetic consistently under-predicts the damage.

Aliasing

A per-line window at fixed phase has a blackout period exactly equal to the raster-line period, which is the natural period of the refresh counter's advance. The same rows therefore sit under the window on *every* sweep and are permanently starved. A coarser epoch whose period-to-sweep ratio is non-integer rotates the blocked set instead, so each row is skipped only occasionally and refreshed on the next pass. The working schedule was a permuted per-line phase using a coprime step.

39. Retention and decay

Refresh suspension	Result
2 ms	pass
4 ms	3 stable failures
8 ms	2 stable failures

With a normal sweep period of ~3.27 ms, the corresponding worst-case row intervals are 5.27 ms (passed) and 7.27 ms (failed). **Retention onset on that machine lies between roughly 5 and 7 ms of row interval.**

Recorded honestly: the 4 ms and 8 ms counts are non-monotonic, which indicates the measurement sits near its noise floor.

Polarity

On the C128, decay was **polarity-locked**: odd addresses railed toward `$FF` — zero-to-one flips only — and even addresses toward `$00`, with 20 of 21 conforming cells. A decayed byte need not reach a rail, because eight independent bit cells can yield a stable mixed value; only bits that *can* decay do so.

THE RULE DOES NOT GENERALISE

The same polarity rule was tested against a corruption mode observed on a C64 and did **not** transfer — 12 conforming against 16 violating. Polarity-locked decay is a signature of one mechanism, not of DRAM corruption in general.

40. Methodology that proved necessary

Several of these were learned the expensive way.

1. **Prove a diagnostic can emit before trusting its silence.** A clean result from an instrument that was never capable of reporting is worthless.
2. **Pulse count is not pulse effectiveness.** Exact accounting proves emission, not effect.
3. **A two-value oracle hides errors.** Any wrong value that happens to equal the expected value is invisible. Use an address-dependent high-entropy pattern.
4. **A repeated identical write pass cannot detect its own losses.** If pass 2 loses a write at an address pass 1 wrote correctly, the byte still holds the right value. Write the complement on the second pass.
5. **A constant-valued region is not an oracle at all** once it already holds that constant.
6. **Read-modify-write on mixed I/O registers is its own fault class.** A bad *read* corrupts a *write*, and the damage persists in hardware.
7. **Watch for scan-order confounds.** When addresses are visited in a fixed order, address and time are the same coordinate; decorrelate them before concluding anything about either.
8. **Intermittent failures make small arm counts worthless.** If a control fails only half the time, four clean arms prove nothing. Saturate the control first, and score by error count rather than pass/fail.
9. **Try a second machine before building a tenth instrument.** A fault that survived nine diagnostic builds on one board was identified in a single boot by running the unchanged image on another. Board-to-board DRAM margin varies more than it is comfortable to assume, and "does this reproduce elsewhere" is the cheapest question available at every stage.

Toward an FPGA Reimplementation

The obstacle has never been that the SuperCPU is complicated. It is that the parts that decide its behaviour cannot be read out. This part sets out what a behavioural reimplementation would have to do, using this document as the specification, and is honest about which parts are solved and which are not.

41. Why it has not been done, and what has changed

The accelerator's glue logic lives in an Altera CPLD — a larger part on V2, which fixed most of V1's bugs. Those devices are read-protected in practice, so nobody has been able to extract the equations and clone the design gate for gate. Every attempt that starts from "dump the CPLD" stops immediately.

A BEHAVIOURAL REIMPLEMENTATION DOES NOT NEED THE EQUATIONS

What a clone needs is not the CPLD's internal logic but its *externally observable contract*: which addresses decode, what each register does on read and on write, when memory is mirrored, where vectors come from, and how the machine is taken over. All of that is documented behaviour, and most of it is now in this document with a source attached.

That reframes the project from reverse-engineering a part to implementing a specification.

What is genuinely still missing is listed in §44. It is a short list, and none of it is locked inside the CPLD.

42. The specification, as it now stands

Collecting the obligations scattered through this document into one place. A design that does all of this is a SuperCPU as far as software can tell.

Obligation	Behaviour	Ref
Bus takeover	Assert /DMA; the host CPU tri-states and stops driving. The VIC-II is untouched and keeps generating the display and refreshing DRAM.	§2
Banking	Emulate the host CPU's \$00/\$01 port and resolve the PLA map internally, since the real port is inside an inert chip.	§2, §5
Execution	65816 at 20 MHz against local SRAM; reads served locally at full speed.	§1, §3
Coherency	Writes go to both local RAM and host DRAM, one host cycle each, because the VIC-II reads DRAM.	§3
Optimization	Four selects at \$D074-\$D077 declaring which region the VIC actually reads, so the rest is never mirrored.	§10
Speed	Three independent 1 MHz requests plus the physical switch; fast only when none asserts. Switch honoured only while the register bank is closed.	§3
Disk	Throttle to 1 MHz for drive access regardless of the speed setting.	§3
Registers	\$D07x write-sensitive switches; \$D0B0-\$D0BF readable status with several writable entries; low three optimization bits OR'd into every read of the block.	§10, §11
Private RAM	\$D200-\$D2FF system, \$D300-\$D3FF user. Read always; write only while the register bank is open, with \$D27E the exception.	§7
ROM shadow	Bank-1 images served into bank 0 per the mirroring table, including the alternate KERNAL that swaps in when the register bank opens.	§6
Vectors	Three-way fetch: bank 0, CPU ROM, or bank 1 at \$017FE4-\$017FFF, per the five conditions.	§6
Detection	\$D0BC bit 7 reads 0; \$D0B0 reports version and mode.	§12
Expansion	Banks \$02-\$F5 user, \$F6-\$F7 relocated system, \$F8-\$FF ROM.	§4, §25

The ROM question, which has a clean answer

SuperCPU DOS is CMD's copyright and cannot be redistributed with a clone. That sounds like a blocker and is not, because the accelerator does not *need* it to function as an accelerator.

The machine's own KERNAL and BASIC can be read off the running host: after a cold start \$01 = \$37 leaves both banked in, so a bus master can simply read \$A000-\$BFFF and \$E000-\$FFFF into its shadow before doing anything else. That is roughly 16,384 host cycles — about 16 ms — and it yields the exact ROM revision fitted to that machine.

A clone built that way accelerates a stock machine with no CMD code at all. What it forgoes is SuperCPU DOS and the JiffyDOS it carries; an owner with their own dump can supply it.

THE ONE THING THAT CANNOT BE SNAPSHOTTED

The character ROM. Exposing it needs CHAREN low, and with the host CPU held off the bus nothing can rewrite its port. This only matters for software that reads the character set *through the CPU* — the VIC-II fetches it directly and the display is unaffected either way.

43. Hardware envelope

The logic is small by modern standards. The difficulty is entirely at the pins.

Element	Requirement	Notes
Fabric	~10k LUT class	A 65816 soft core is roughly 4-8k LUTs; bus mastering, register file and the mirror engine add a few thousand. Comfortably inside a mid-range ECP5, Cyclone or Spartan part. Capacity is not the constraint.
Main memory	128 KB minimum, 256 KB for a 128-class design	Async SRAM at 10-15 ns is the simplest match and mirrors the original. 20 MHz means a 50 ns access budget, so this is not demanding. Block RAM alone is tight at 1 Mbit.
Expansion	up to 16 MB	The original used a fast-page-mode SIMM because that is what 1996 offered. A modern design would use SDRAM and keep the bank map identical, since the map is what software sees.
Level shifting	~30 signals, 5 V ↔ 3.3 V, bidirectional on the data bus	The real work. The host bus is 5 V TTL and FPGAs are not 5 V tolerant.
Clocking	Local PLL, plus a PHI2 edge detector	Everything is timed <i>against the host</i> , not against a free-running oscillator. PHI2 is about 1.023 MHz NTSC and 0.985 MHz PAL.
Connector	44-pin expansion port	Address, data, R/W, PHI2, /IRQ, /NMI, /RESET, /DMA, BA, /GAME, /EXROM, /ROML, /ROMH, /IO1, /IO2, DOTCLOCK.
Power	Consider not taking it from the port	The original already exceeded the port's nominal current specification, and Initialization Error #6 exists precisely because of it — §30. A clone with its own supply removes an entire class of field failure.

WHERE AN FPGA WINS DECISIVELY

The bus state machine becomes hardware. A software bus master — which is what this project's own emulator is — must hit sub-microsecond deadlines from a CPU that has caches, interrupts and a scheduler, and a single instruction-fetch stall inside a bus window corrupts a transfer. In an FPGA the timing is structural: it either meets the constraint or the tools say so at build time.

That advantage is the strongest argument for the approach, and it is worth more than the logic capacity.

44. The hard parts

Four, in increasing order of how much they should worry a designer.

1. The register contract has gaps

Most of it is documented, but not all. `$D078` takes a SIMM configuration value whose encoding is not published. Several `$D0Bx` entries are decoded with no known flags of their own. The V2 enhanced optimization register is documented in one contemporary article and not independently confirmed. A clone can behave sensibly at those addresses without knowing exactly what the original did, but it will not be bit-exact, and software that probes them will find out.

2. Bus timing is where reimplementations die

This is not a theoretical caution. The write timing that works differs between machines and between PAL and NTSC; R/W has to be driven before the address latch is enabled; and older VIC-II revisions need more margin than newer ones.

A cautionary tale from the same work, worth passing on. A read-sampling calibration in this project reported that it *could not locate a stable window at all* on two different hosts, which was read as an alarming hardware finding for six build cycles. It was an artefact of the instrument: the scan scored DRAM reads and one VIC register read together in a single zero-error criterion, so a single always-failing check vetoed every otherwise-clean sample point. Score device classes separately, and log the value read rather than only pass/fail — a calibration that cannot distinguish "no valid timing exists" from "one check is broken" will send you a long way in the wrong direction.

DESIGN THE TIMING TO BE ADJUSTABLE, NOT FIXED

Any fixed set of setup and hold constants will work on the machine it was tuned on and fail somewhere else. A clone should calibrate at power-on against the host it is actually in — sweep the sampling point, score it, and keep the widest passing window — and expose the result so a failure is diagnosable rather than mysterious.

3. Holding /DMA continuously has consequences the original may have managed

Part X records DRAM disturbance measured under sustained expansion-port DMA on real hardware: corruption confined to specific address classes, appearing only while the VIC is actively fetching, and stopping when physical bus access stops. Whatever the mechanism turns out to be, a clone holds /DMA in exactly the same way and inherits exactly the same question.

Resolved, and the answer was the reassuring one. The same diagnostic image was run unchanged on a second C64. Machine A produced 64 stored errors at exactly `$xxF8/$xxF9` in every arm; machine B produced **zero**, in all twelve arms, including the identical data pattern that machine A failed on. The failures were also data-dependent: uniform `$00`, uniform `$FF` and alternating patterns were clean at the same addresses, and only mixed high-entropy content failed.

That is marginal DRAM on one board, not a property of holding /DMA. A clone does not inherit it.

THE GENERAL LESSON IS STILL WORTH KEEPING

Two adjacent rows failing across all columns, only under mixed data, is a plausible signature of neighbour coupling in an ageing DRAM — and a design that holds the bus for seconds at a time will surface exactly this kind of marginality on machines that have it. Expect field reports that look like accelerator bugs and are not, and give the firmware a way to tell the difference: **the same image on a second machine** answered in one boot what nine diagnostic builds could not.

4. The C128 needs the socket

A 128-class design is not a cartridge-only proposition. Bank `$00` must track the current C128 RAM bank selection, and that selection lives inside the 8722 where the expansion port cannot see it — hence the MMU adapter, §37. Any 128 reimplementation needs the same connection, and the first question to answer about it is whether the adapter taps signals or intercepts them.

45. A staged plan, and how to know each stage works

The value of the register contract is that it is testable from BASIC on the target machine, so each stage has an acceptance test before the next begins.

Stage	Deliverable	Acceptance
1	Bus master only: assert /DMA, read and write host memory, no CPU	Read the KERNAL off the machine and checksum it against a known dump. If this is not solid, nothing above it will be.
2	65816 core running from local SRAM, ROM snapshotted from the host	The machine boots to READY at 1 MHz. No acceleration yet — correctness first.
3	Write-through mirroring	The display stays correct while a program writes to screen memory. This is where flicker and tearing appear if the raster gating is wrong.
4	Speed control and the register block	The BASIC status readout in §24 prints the right thing, and the benchmark in §25 shows the expected ratio.
5	Optimization modes, private RAM, vector reroute	Software that probes \$D0Bx behaves; GEOS runs.
6	Expansion memory	Detection through the allocation pointers at \$00D27C-\$00D27F, then a long-addressed read/write sweep across every fitted bank.
7	C128 native mode	Only after the adapter question of §37 is settled.

Two things make this more tractable than it looks. The compatibility rules in Part VII double as a software test suite — each one names a behaviour some real program depends on. And a software emulator of the same device, however slow, is a reference model to diff against: when the FPGA and the model disagree about what a register returns, one of them is wrong and the disagreement is the bug report.

Reference Material

A. Timing and raster arithmetic

Quantity	Value
PAL C64 cycle	1016 ns (985,248 Hz)
NTSC C64 cycle	978 ns (1,022,727 Hz)
One C64 bus access	~1 µs

Standard	Lines	Cycles/line	Cycles/frame
PAL	312	63	19,656
NTSC 6567R56A	262	64	16,768
NTSC 6567R8	263	65	17,095

A badline steals 40–43 cycles, so eight consecutive lines cost about $63 \times 7 + 23 = 464$ cycles rather than 504.

Machine detection heuristics

- **Machine running:** time 1000 C64 cycles and expect 1.2–1.6 M ARM cycles on a Pi. Outside that window the machine is not up.
- **PAL vs NTSC:** sample `$D012` plus `$D011` bit 7 for a frame and keep the maximum. PAL reaches 311, NTSC only 261 or 262.
- **C64 vs C128:** `$D030` exists on a C128 and reads `$FF` on a C64.

B. SuperCPU DOS and the startup animation

The C64 startup animation is stored in `scpu-dos-2.04.bin` as a relocatable blob. Traced from the reset path of the 128 KB image with SHA-1 `6aa529a7...7f5`:

- The reset vector at ROM offsets `$FFFC`–`$FFFD` is `$FE00`.
- `$FE00` selects the machine path; the C64 path reaches the long-ROM startup code through `$FC90` (`JML $F800FC`).
- At ROM offset `$01E0`, native-mode code performs an `MVN` of `$0E01` bytes from `$F8:D000` to bank-0 RAM `$0E00`, then `JML $001400`.

```
PHB / CLC / XCE / REP #$30
LDA #$0E00          ; MVN count minus one
LDX #$D000          ; source offset in bank $F8
LDY #$0E00          ; destination offset in bank $00
MVN $F8,$00
SEC / XCE / PLB
JML $001400
```

RAM \$1400 is a trampoline to the animation body. It is not an ordinary subroutine — it continues to the appropriate KERNAL cold-start path rather than returning to its caller. Calling \$1400 a second time is not sufficient: the animation is self-modifying and leaves its working copy changed, so re-entering the dirty copy produces a partially drawn static screen.

B2. The two DOS images, compared byte by byte

Everything in this subsection was established by direct examination of the two circulating 128 KB images, not from documentation. It is reproducible: the method is a block-occupancy scan, a per-block difference count, and a screen-code and PETSCII string sweep.

	DOS 1.40	DOS 2.04
Size	131,072 bytes	131,072 bytes
SHA-1	3422a773...eab7a	6aa529a7...727f5
Populated	low 64 KB only — \$00000-\$0BFFF and \$0D000-\$0FFFF; the upper 64 KB is entirely \$FF	all 128 KB — 32 of 32 blocks carry data
Banner, ROM \$0E480	SUPERCPU DOS 1.40 (C)1996 CMD	SUPERCPU DOS 2.04 (C)1996 CMD
Second banner	none	\$1F6CB: ...MB SUPERCPU DOS 2.04 (C)1997 CMD
Bank-0 reset vector	\$FFFC → \$FC90	\$FFFC → \$FE00
Upper-bank vectors	absent (all \$FF)	present — reset \$FF3D

How much is actually shared

Less than the common 1996 copyright line suggests. At 4 KB granularity only **two of thirty-two blocks are byte-identical**. At 256-byte granularity, **126 of the 256 pages in the low 64 KB match** — so roughly half the C64-side image is common code and half was rebuilt.

The largest contiguous identical run is \$06D00-\$07EFF, which is the RL DOS region: both images carry the same *CMD RL DOS COPYRIGHT 1990 CREATIVE MICRO DESIGNS, INC* string at \$07EA0 and the same error vocabulary — SYNTAX, DIRECTORY, FORMAT, HARDWARE, SYSTEM, CONTROLLER and I/O ERROR. The disk-side DOS was inherited essentially untouched; the accelerator-side code was not.

What 2.04 has that 1.40 does not

Two findings put the C128 support beyond inference.

The upper bank carries a C128 system. 2.04 contains COMMODORE copyright strings at \$141BF, \$141F2 and \$17FC8 and a MICROSOFT string at \$14220. 1.40 contains none of them anywhere — its only matches for BASIC are the C64 banner and DOS text.

The boot self-test speaks to the MMU. The routine immediately preceding the initialization-error message differs structurally between the images, and not cosmetically:

Boot check	DOS 1.40	DOS 2.04
Processor mode	stays in emulation mode	CLC / XCE — enters native mode
Index width	8-bit	REP #10 — 16-bit
Memory touched	\$01, then a long-addressed pattern test	\$01, then \$FF00 — the C128 MMU configuration register — before its pattern test
Test loop	fixed long addresses, EOR #\$FF verify	indexed long addressing across a range, EOR #\$FF verify

A write to `$FF00` in the first few hundred bytes of the reset path is not something a C64-only ROM does. It is the always-visible C128 configuration register from §35, and 2.04 configures it before it trusts memory.

The initialization error message

The message appears exactly once in each image, stored as **screen codes** rather than PETSCII — so it is written straight into screen memory, which is what lets it appear before any KERNAL output path is trusted.

	DOS 1.40	DOS 2.04
Message text	SUPERCPU INITIALIZATION ERROR:	
ROM offset	\$00326	\$006AA
Digit table	\$00345	\$006C9

THE CODES ARE HEXADECIMAL

Immediately after the trailing space sits a **sixteen-entry lookup table** holding the screen codes for 0 – 9 then A – F. The display routine indexes it per nibble, so the number after the colon is a hex byte, not decimal — and codes above 09 will appear as letters.

What the codes mean

Only one is documented, and it is the one worth knowing:

Code	Meaning	Source
06	Inadequate power supply. See §30 — this is a supply fault, not a ROM, machine or DOS-version fault.	CMD SuperCPU FAQ
others	<i>not established</i>	—

WHY THE REST ARE NOT LISTED HERE

There is no table of causes in either image — the ROM stores the message and a hex-digit converter, and nothing that maps a number to a description. Recovering the full set means disassembling every path that reaches the display routine and working out what each one was testing.

That work has not been done, so this reference lists one code rather than eight plausible ones. An invented table would be worse than no table, because it would be believed.

C. Sources, and what this document omits

Primary sources

- CMD *SuperCPU 128 V2 User's Guide* and *SuperRAM Installation Guide and User's Reference*.
- Doug Cotton, *The SuperCPU Steps Up to Version 2*, *Commodore World* issue 22 — the v2 architecture and the complete `$D0B3` encoding.
- The Go64! SuperCPU tutorial series, parts 1–10, with its memory-locations and memory-mirroring reference pages.
- Western Design Center, *65c816 Assembly Language Standard* — the assembler syntax, forced-mode suffixes, typed equates and directive set summarised in Part V. The explanations and all code examples in that part are this document's own.
- The SuperCPU FAQ at `supercpu.cbm8bit.com` — power requirements, cartridge compatibility and the meaning of Initialization Error #6, in §30.
- Malte Mundt (ThunderBlade / Protovision), *Learn programming in a SuperCPU-compatible way*, 1999 — the compatibility rules in Part VII.
- *C=Hacking* issue 17 — the software-repair catalogue and diagnostic method in §29.
- *A solved secret*, Go64! 03/98 — the `$D019` read-modify-write cycle counts.
- VICE's SCPU64 implementation (`scpu64mem.c`), which is exercised by real SuperCPU software and which corrected several long-standing documentation errors.
- The Commodore 128 service manual and the 8722 MMU programming notes.
- Original measurements on physical C64 and C128 hardware, recorded in Part VIII.

Where documentation and implementation disagreed

Several entries in the register tables were originally derived from manuals and community documentation and were wrong. `$D078` is SIMM configuration rather than an optimization mode; `$D079` is a mirror of `$D07B`; `$D072` and `$D073` were missing entirely; and the `$D0Bx` block is not one mirrored status byte but sixteen distinct addresses, several of them writable. Where documentation and a working implementation disagree, this compendium follows the implementation.

A note on the cover

The badge on the title page is the **original SuperCPU artwork** (`Docs/SuperCPU-logo.gif`), tonally inverted to black-on-white, trimmed to its bounding box and rescaled for print. Nothing about the letterforms has been redrawn.

"CMD", "SuperCPU" and "SuperRAM" are the marks of Creative Micro Designs. This is an independent technical reference with no affiliation to, or endorsement by, the trademark holder.

Deliberately omitted

This document covers the CMD SuperCPU devices themselves. Material specific to the SCPU-EMU project — its build process, host test suite, source-tree layout, roadmap and RAD Expansion Unit GPIO assignments — is not reproduced here, since it describes an implementation of the device rather than the device. Those remain in the project's own documentation.

Compiled from the SCPU-EMU project documentation, 11 August 2026. Hardware measurements in Part X were taken on one physical C64 and one physical C128; machine-to-machine margins differ.